

CEN 116 EXERCISE

- 1) Given the following function for fun and the definitions shown below

```
int fun (int *, int *);
```

```
int i = 2, j = 4;  
int *pi = &i;  
int *pj = &j;
```

indicate whether each of the following calls to fun is valid:

- a) i=fun(2,10); // is not valid
- b) j = fun (&i, &j); //is valid
- c) i = fun (pi, &j); //is valid
- d) j = fun (i,j); //is not valid
- e) fun (pi, pj); //is valid

- 2) What is the output from the following program?

OUTPUT: 1. 4 17 9
2. 35 17 14 19 8 20 11

```
#include <stdio.h>  
int fun (int *px, int y ,int *pz) {  
  
    int a = 5;  
    int *p;  
    printf("1. %d %d %d\n", *px,y,*pz);  
  
    for ( p = pz; p < pz+5; ++p)  
  
        *p = a+*p;  
  
    return (*px+*pz+y);  
}  
int main () {  
  
    int a = 4, b = 17, c[5] = {9,14,3,15,6};  
  
    a = fun(&a,b,c);  
    printf("2. %d %d %d %d %d %d %d\n", a,b,c[0],c[1],c[2],c[3],c[4]);  
  
    return 0;  
  
}
```

3) What would be printed from the following block:

OUTPUT:

6 4
1 2
6

```
#include <stdio.h>

void print0(int *x)
{
    printf("\n%d %d",x[2],*x+1);
}
void printT (int *x)
{
    printf("\n%d %d",x[2], *(x+1));
}
void printTh(int *x)
{
    printf("\n%d\n",*x);
}
int main()
{
    int num[5] = {3,5,6,2,1};
    print0(num);
    printT(num+2);
    printTh(&num[2]);
    return 0;
}
```

4) What would be printed from the following block:

p: 7 6 9

q: 4 7

t: 5 7 7

```
#include <stdio.h>
int main (void)
{
    int x [2][3] = {{4,5,2},{7,6,9}};
    int (*p) [3] = &x[1];
    int (*q) [3] = x;
    int * t [3] = {&x[0][1],p[0],q[1]};
    printf("\np: %d %d %d\n",(*p)[0],(*p)[1],(*p)[2]);
    printf("\nq: %d %d\n", *q[0],*q[1]);
    printf("\nt: %d %d %d\n", *t[0],*t[1],*t[2]);
    return 0;
}
```

5) Write a C program that reads integers from the keyboard and place them in an array. The program then will sort the array into ascending and descending order and print the sorted lists. The program must not change the original array or create any other integer arrays. (The size of the array will be read from the keyboard). (Your program will include two pointer arrays)

```
#include <stdio.h>
#include <stdlib.h>

int ** ascending(int *p,int n)
{
    int **k,i,j,temp;

    k = (int **) calloc(n,sizeof(int *));
    for (i = 0; i < n; i++)
    {
        k[i] = (int *) calloc(1,sizeof(int));
        *k[i] = p[i];
    }
    for (i = 0; i < n; i++)
    {
        for (j = i+1; j < n; j++)
        {
            if(*k[i] > *k[j])
            {
                temp = *k[j];
                *k[j] = *k[i];
            }
        }
    }
}
```

```

        *k[i] = temp;
    }
}
return k;
}

int ** descending(int *p, int n)
{
    int **k, i, j, temp;

    k = (int **) calloc(n, sizeof(int *));
    for (i = 0; i < n; i++)
    {
        k[i] = (int *) calloc(1, sizeof(int));
        *k[i] = p[i];
    }
    for (i = 0; i < n; i++)
    {
        for (j = i+1; j < n; j++)
        {
            if(*k[i] < *k[j])
            {
                temp = *k[j];
                *k[j] = *k[i];
                *k[i] = temp;
            }
        }
    }
    return k;
}

int main (void)
{
    int *p, i, j, **q, **t;

    printf("Array Size:");
    scanf("%d", &j);

    p = (int *) calloc (j, sizeof(int));

    for (i = 0; i < j; i++)
        scanf("%d", &p[i]);

    q = ascending(p, j);
    t = descending(p, j);
    printf("D\t O\t A\n");
    for (i = 0; i < j; i++)
    {
        printf("%d\t %d\t %d\n", *t[i], p[i], *q[i]);
    }
    free(p);
    return 0;
}

```

- 6) **Write a C program to answer inquiries about student data. Student data contains id, name, surname and 5 quiz scores. Your program displays a menu providing the following choices:

- 1) Search for a student with name, surname or id.
- 2) Print out the average score of students
- 3) Add a new student data
- 4) Print out the average and standard deviation of each quizzes.

You will use struct, dynamic memory allocation, string functions, and function pointer in your code.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

typedef struct{
    int id;
    char *name;
    char *surname;
    int quiz[5];
}STUDENT;

STUDENT *std;

char temp[50];

int totals;

void Add(){

    int j, i, n,nlen;
    static int count=0;

    printf("How many student do you want to enter:");
    scanf("%d",&n);
    std = (STUDENT*)realloc(std,(n+count)*sizeof(STUDENT));
    printf("%d student will registered\n",count+1);
    for (i = 0; i < n; i++)
    {

        printf("ID:");
        scanf("%d", &(std + count)->id);
        printf("How many character will you input for Name:");
        scanf("%d",&nlen);
        (std + count)->name = (char*)calloc(nlen + 1,
sizeof(char));
        scanf(" %s", (std+count)->name);
        printf("How many character will you input for
Surname:");
        scanf("%d",&nlen);
        (std + count)->surname = (char*)calloc(nlen+1,
```

```

sizeof(char));
    scanf(" %s", (std+count)->surname);
    printf("Enter quizzes:\n");
    for (j = 0; j < 5; j++)
    {
        printf("%d quiz=>", j+1);
        scanf("%d", &(std + count)->quiz[j]);
    }

    count++;
    totals = count;
}
getchar();

system("cls");
}

void Search()
{
    char name[100];
    int ch, id ,i, j;

    printf("1.Name\t2.Surname\t3.ID\n");
    fflush(stdin);
    scanf("%d", &ch);
    fflush(stdin);
    printf("TOTAL STUDENT %d", totals);
    switch (ch)
    {
        case 1:
            system("cls");
            printf("\n\nEnter student name:");
            scanf(" %s", name);

            for (i = 0; i < totals; i++)
            {
                if(strcmp(name, (std+i)->name)==0)
                {
                    printf("%d. Student:", (i+1));
                    printf("ID: %d\n", (*(std+i)).id);
                    printf("NAME: %s\n", (*(std+i)).name);
                    printf("SURNAME:
%s\n", (*(std+i)).surname);
                    for (j = 0; j < 5; j++)
                    {
                        printf("QUIZ %d: %d\n", j + 1, (std +
i)->quiz[j]);
                    }
                }
            }
        }
    }
}

```

```

        break;
    case 2:
        system("cls");
        printf("\n\nEnter student surname:");
        scanf(" %s", name);

        for (i = 0; i < totals; i++)
        {
            if(strcmp(name, (std+i)->surname) == 0)
            {
                printf("%d. Student:", (i+1));
                printf("ID: %d\n", (*(std+i)).id);
                printf("NAME: %s\n", (*(std+i)).name);
                printf("SURNAME:
%s\n", (*(std+i)).surname);
                for (j = 0; j < 5; j++)
                {
                    printf("QUIZ %d: %d\n", j + 1, (std +
i)->quiz[j]);
                }
            }
        }

        break;
    case 3:
        system("cls");
        printf("\n\nEnter student id:");
        scanf("%d", &id);

        for (i = 0; i < totals; i++)
        {
            if(id == (std+i)->id)
            {
                printf("%d. Student:", (i+1));
                printf("ID: %d\n", (*(std+i)).id);
                printf("NAME: %s\n", (*(std+i)).name);
                printf("SURNAME:
%s\n", (*(std+i)).surname);
                for (j = 0; j < 5; j++)
                {
                    printf("QUIZ %d: %d\n", j + 1, (std +
i)->quiz[j]);
                }
            }
        }
        break;
    }
    getchar();

```

```

        system("cls");
    }

    void Average(){
        float avg=0;
        int i,j;
        for (i = 0; i < totals;i++)
        {
            avg =0;
            for (j = 0; j < 5; j++)
            {
                avg += (std + i)->quiz[j];
            }
            printf("%d. Student Average Score is
%f\n", (i+1), (avg/5));
        }

        getchar();
        system("cls");
    }
    void AverageDeviation(){
        float avg[5] = {0}, dev[5]={0};
        int i, j;

        for (j = 0; j < 5; j++)
        {
            for (i = 0; i < totals; i++)
            {
                avg[j] += (std + i)->quiz[j];
            }
            avg[j]/=totals;
            printf("AVERAGE OF %d. QUIZ = %f\t STANDARD
DEVIATION OF %d. QUIZ ", (j+1), avg[j], (j+1));
            for (i = 0; i < totals; i++)
            {
                dev[j] += pow (((std+i)->quiz[j]-avg[j]),2);
            }
            dev[j] /=totals;
            printf("= %f\n", sqrt(dev[j]));
        }

        getchar();
        system("cls");
    }

    void EXIT()
    {
        free(std);
    }

```

```

        exit(0);
    }

    int main()
    {
        void(*Menu[])() = { Add, Search, Average,
        AverageDeviation, EXIT};
        int ch;
        std = (STUDENT*)calloc(1, sizeof(STUDENT));
        while (1){

            printf("\n\n1-Add\t2-Search\t3-Average\t4-
AverageDeviation\t5-EXIT\n\n");
            printf("Enter a number from menu:");
            scanf("%d", &ch);
            system("cls");
            Menu[ch- 1]();
        }

        return 0;
    }

```

- 7) While gets() function reads until EOF or \n has been inputted, scanf read until any whitespace has been encountered. However, gets() function may generate buffer overflow errors to avoid fgets() should be used.

- 8) Write a C program to count the number of '.' and ',' in a given text.

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main()
{
    char * text;
    char * textt;
    int n,npoint=0,ncomma=0;
    char * token, *ctoken;

    printf("The length of your text\n");
    scanf("%d",&n);
    fflush(stdin);
    text = (char *) calloc ((n+1),sizeof(char));
    textt = (char *) calloc ((n+1),sizeof(char));
    printf("Enter your text:\n");
    scanf("%s[^\n]",text);
    fgets(text,n,stdin);

```

```

    strcpy(textt, text);

token = strtok(text, ".");
while(token)
{
    printf("%s\n", token);
    npoint++;
    token = strtok(NULL, ".");
}
printf("Number of . = %d ", npoint);

ctoken = strtok(textt, ",");
while(ctoken)
{
    printf("%s\n", ctoken);
    ncomma++;
    ctoken = strtok(NULL, ",");
}
printf("Number of , = %d ", ncomma);

free(text);
free(textt);
return 0;
}

```

- 9) Find the value of *x, *(x+1), and *(x+4) for the following declaration:

```
char *x = "The life is beautiful";    T, h, l
```

- 10) Find the value of *y, *(y+1) and *(y+4) for the following program segment:

```
char x [] = "The life is beautiful";
char* y = &x[3];                ' ', l, e
```

- 11) Pointers must point to some variables. We can do this by assigning address of a variable, which is located at stack or allocating memory dynamically in heap. In that example, pointer is used before initialization.

- 12) What would be printed from the following program block?

```

{
    char * s1 = "abefgnpanm";
    char * s2 = "ab";
    char * s3 = "pan";
    char * s4 = "bef";
    char * s5 = "panam";
    int d1, d2, d3, d4;
    d1 = strstr(s1, s2);
    d2 = strstr(s1, s3);
}

```

```

    d3 = strstr(s1,s4);
    d4 = strstr(s1,s5);
    printf("%d\n",d1);
    printf("%d\n",d2);
    printf("%d\n",d3);
    printf("%d\n",d4);
}

```

strstr searches str2 in str1 and counts the number of matching elements until it finds unmatched character.

Output : 2
1
0
1

13) What would be printed from the following program block?

OUTPUTS : GOOD

UGLY

L

WICKED

NICE

C

E

{

```

    char * a[5] ={"GOOD","BAD","UGLY","WICKED","NICE"};
    printf("%s\n", a[0]);
    printf("%s\n",*(a+2));
    printf("%c\n",*(a[2] + 2));
    printf("%s\n",a[3]);
    printf("%s\n",a[4]);
    printf("%c\n",*(a[3] + 2));
    printf("%c\n", *((a+4)+3));

```

}

14) Build a link list including the given elements: 111 222 333 444 555 666 777 and insert 50 into the end of the list and delete 333 from the list.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct LLIST
```

```
{
```

```
    int num;
```

```
    struct LLIST * next;
```

```
}NODE;
```

```

NODE *first = NULL, *last=NULL;

void AddNode(NODE *k)
{
    if(first == NULL) // if list is empty
    {
        first = k;
        last = k;
        last->next = NULL;
    }
    else
    {
        last->next =k;
        last = k;
        last->next = NULL;
    }
}

void DeleteNode(int n)
{
    NODE *p, *previous =NULL;

    p = first;
    while (p)
    {
        if(p->num == n)
            break;
        previous = p;
        p = p->next;
    }
    if (p != NULL) //we find the element
    {
        if(p == first) // p is the first element
        {
            if(first == last) //only one element that is
p
            {
                first = NULL;
                last = NULL;
            }
            else //there are other elements we delete
the first
            {
                first = first->next;
            }
        }
        else //p is not first element
        {
            previous->next = p->next;
            if (p == last) //if p is the last element
                last = previous;
        }
    }
}

```

```

        free(p);
    }
}
int main()
{
    int i;
    NODE *k,*temp;

    for (i = 0; i < 7; i++){

        k = (NODE *) malloc(sizeof(NODE));
        k->num = (i+1)*111;
        AddNode(k);

    }
    k = (NODE *) malloc(sizeof(NODE));
    k->num = 50;
    AddNode(k);
    DeleteNode(333);
    temp = first;
    i = 0;
    while(temp)
    {
        printf("%d. element %d\n", (i+1), temp->num);
        temp = temp->next;
        i++;
    }
    free(k);

    return 0;
}

```